The Key Insight: Threads vs Work

In a traditional (blocking) approach, one thread does all the work AND waits for the broker ACK before returning. In this implementation, threads hand off work and return to the pool immediately -- the actual I/O wait happens on a Kafka-managed I/O thread, not on a Tomcat Servlet thread.

Request Flow -- Step by Step

```
  HTTP Request arrives
         |
         v
+--------------------------+
|   Tomcat / Servlet       |  Thread T1 picked from the Tomcat thread pool
|   Thread Pool            |
+-----------+--------------+
            |
            v
+--------------------------+
|       Controller         |  T1: @Valid validates request, calls service.
|  (CF<ResponseEntity>)    |  Returns CF to Spring MVC -> Spring registers
|                          |  a callback, RELEASES T1 back to pool.
+-----------+--------------+
            | (T1 is now FREE -- no blocking)
            v
+--------------------------+
|         Service          |  Still on T1: sets EventType synchronously.
|   (CF<LibraryEvent>)     |  Calls producer.sendLibraryEvent(), chains
|                          |  .thenApply() + .exceptionally() -- only
|                          |  REGISTERS callbacks, no work executes yet.
+-----------+--------------+
            |
            v
+--------------------------+
|         Producer         |  T1: builds Message, kafkaTemplate.send().
|    (CF<SendResult>)      |  Kafka enqueues to internal buffer, returns CF
|                          |  immediately. .whenComplete() registers a
|                          |  logging callback. T1 returns to pool.
+-----------+--------------+
            |
   . . . T1 is FREE. No thread is waiting . . .
            |
   [ Later: Kafka I/O thread receives broker ACK ]
            |
            v
+--------------------------+
|    Kafka I/O Thread      |  Completes the CompletableFuture.
|  (NOT from Tomcat pool)  |  Callback chain fires in order:
|                          |   1. whenComplete -> logs success / failure
|                          |   2. thenApply    -> result -> event
|                          |   3. thenApply    -> event -> ResponseEntity
+-----------+--------------+
            |
            v
+--------------------------+
|   Spring MVC Async       |  Receives completed ResponseEntity.
|     Dispatcher           |  Writes HTTP response back to client.
+--------------------------+
```

Callback Chain Diagram

```
kafkaTemplate.send()  -----> CF<SendResult>
                                  |
                        broker ACK (Kafka I/O thread)
                                  |
             whenComplete  -----> logs partition + offset        [Producer]
                                  |
             thenApply     -----> SendResult  ->  LibraryEvent    [Service]
                                  |
             thenApply     -----> LibraryEvent -> ResponseEntity  [Controller]
                                  |
             Spring MVC    -----> writes HTTP 201 response

    (if broker rejects / timeout)
                                  |
             whenComplete  -----> logs error
                                  |
             exceptionally -----> throws LibraryEventPublishException
                                  |
             Spring MVC    -----> @ControllerAdvice  ->  HTTP 500
```

Code Walkthrough: thenApply / exceptionally Chain

```
return libraryEventProducer.sendLibraryEvent(event)   // (1)
       .thenApply(result -> event)                    // (2)
       .exceptionally(ex -> {                         // (3)
           throw new LibraryEventPublishException(
               "Unable to publish ADD library event",
               ex.getCause()                          // (4)
           );
       });
```

(1) sendLibraryEvent(event) -- returns CF<SendResult> immediately. No waiting. The future completes later when the Kafka I/O thread gets the broker ACK.

(2) .thenApply(result -> event) -- registers a transformation: when the future completes successfully, ignore the SendResult and return the original event object. Type changes: CF<SendResult> -> CF<LibraryEvent>.

(3) .exceptionally(ex -> { throw ... }) -- registers an error callback: if the future completes exceptionally, re-wrap the cause as LibraryEventPublishException and keep the future failed so @ControllerAdvice can handle it.

(4) ex.getCause() -- the ex inside exceptionally is always a CompletionException (Java's internal wrapper). The real error is one level deeper. Using getCause() avoids wrapping CompletionException inside LibraryEventPublishException.

Why This Matters Under Load

Scenario	Blocking (old)	Async (new)
Threads per request	1 held until broker ACK	~0 (released in microseconds)
100 req, 50ms Kafka latency	100 threads waiting	Near 0 threads waiting
Throughput ceiling	Thread pool size (~200)	Kafka I/O throughput
Tomcat thread usage	High -- proportional to Kafka latency	Minimal -- only for request parsing

A Tomcat default pool has ~200 threads. In the blocking approach each request held a thread for the full Kafka

Why it's End-to-End Asynchronous

round-trip (5-50ms). In the async approach threads are returned to the pool in microseconds, allowing the same 200 threads to handle thousands of concurrent in-flight Kafka requests.